

MSME Subsidy-Capture Agent — V1 Architecture & Feature Specification

Status: Draft for production team review **Scope:** Version 1 only. This is deliberately narrow. Read Section 0 before anything else.

0. Scope Note — Read This First

This document specifies **one product**: an AI agent that sits inside a small CA firm or MSME compliance consultancy, watches a client's registered business data, matches them against government subsidy and scheme rules, and helps the CA assemble and file the claim — priced as a success fee on money actually recovered for the client.

It deliberately does **not** include: compliance-deadline monitoring, tender/GeM matching, procurement, a "business twin" memory layer, community features, multi-agent negotiation, or a knowledge graph. Those were explored and are listed in **Section 13 (Phase 2+ Backlog)** for visibility only. If a capability isn't described in Sections 1-12, it is explicitly **not** in this build.

The reason this note exists at all: the product concept behind this went through three rounds of scope expansion before landing here. Treat every future "wouldn't it be cool if we also—" as a Section 13 candidate, not a v1 change, unless Section 3's success metrics have already been hit.

1. Problem Statement

India has 63-64 million MSMEs and a \$530B+ MSME credit/subsidy gap, but most first-time business owners can identify only one or two schemes they're eligible for, out of dozens that may apply to their state, sector, and size. Government scheme-discovery portals (Jan Samarth, Udyam, CHAMPIONS) exist but are fragmented across departments and rarely cross-matched to a specific business's full profile. Meanwhile, India's ~100,000 registered CA firms — 72%+ of them single-partner practices — already hold the exact business data (Udyam registration, GST returns, turnover, employee count) needed to do this matching, but have no time or tooling to do it proactively for every client.

The gap: nobody is turning "your client already qualifies for ₹X" into a filed, disbursed claim, systematically, for the CA who already serves that client.

2. Target User & Buyer

Role	Who	Relationship to product
Primary buyer & user	CA / compliance consultant at a small-to-mid firm (1-10 partners)	Logs in, manages clients, reviews AI matches, approves filings
Data subject (indirect)	The MSME business owner	Never needs to use the product directly. Their data enters via the CA, from documents the CA already holds.
Internal ops (v1, small team)	You / a compliance analyst	Manually verifies scheme data, handles edge cases the AI flags as low-confidence

Why the CA and not the MSME directly: the MSME owner has near-zero willingness to pay for standalone software and no time to learn a new tool. The CA already has the client relationship, the underlying documents, and a billing relationship to attach a success fee to.

3. V1 Scope & Success Metrics

Geographic/category scope (fill in before build starts): one state, one or two scheme categories (e.g., CGTMSE collateral-free credit guarantee + PMEGP margin-money subsidy, or MUDRA + a state-specific capital subsidy). Do not launch multi-state or multi-category — expand only after Gate 1 below is hit.

Gate 1 — Prove the wedge (target before any new agent or feature is discussed):

- 30-50 CA/consultancy firms onboarded and actively using the dashboard
- ≥60% of matched schemes result in an actual filed application within 30 days
- At least one full cycle of disbursement → success fee collected, for a meaningful number of clients
- Firms still logging in weekly at day 60 without being chased

If these aren't hit, the answer is to fix this product, not add a second one.

4. User Personas

"Priya, CA, 3-partner firm, Tier-2 city" — Handles GST/ITR filing for ~120 small business clients. Knows two or three schemes well because she's used them before; assumes the rest don't apply to her clients or doesn't have time to check. Wants a way to generate extra billable value from clients she already has, without extra manual research.

"Ramesh, machine-parts manufacturer, Priya's client" — Never logs into anything new. Gives Priya his documents once a year. Wants: money he's entitled to, with the least possible paperwork on his end.

5. System Architecture

5.1 High-Level Flow



5.2 Components

1. **Client Data Ingestion** — document upload (PDF/photo) + OCR extraction for Udyam certificate, GST returns, financial statements. Manual field entry as fallback.

2. **Business Profile Engine** — normalizes extracted data into a structured client profile (see Section 6).
 3. **Scheme Knowledge Base** — the single most important piece of infrastructure in this product. A maintained, versioned database of scheme rules (central + one state), each entry sourced to an official document with a "last verified" date. This is retrieved from, never generated from model memory.
 4. **Eligibility Matching Agent** — RAG-based: retrieves relevant scheme entries from the Knowledge Base, compares against the structured client profile, and produces a match only when it can cite the specific clause justifying it.
 5. **Application Assembly Agent** — pre-fills known fields into the scheme's application format, drafts any required narrative sections, and generates a checklist of missing documents.
 6. **Human Review & Approval Layer** — mandatory gate. No match is shown to the client, and no application is filed, without explicit CA sign-off. This is a product requirement, not just a UI nicety — see Section 7.
 7. **Filing & Tracking** — where an API exists (e.g., Udyam), submit programmatically; otherwise generate a submission-ready package and let the CA file manually, tracked as a status.
 8. **Success Fee Billing Engine** — tracks confirmed disbursement, calculates the agreed success fee, generates an invoice.
 9. **CA Dashboard (web app)** — the primary interface. See Section 11.
 10. **Notification Layer** — email + optional WhatsApp digest for new matches, deadlines, and status changes.
-

6. Data Model (Core Entities)

```
BusinessProfile
- id, ca_firm_id
- business_name, udyam_number, gstin
- sector (manufacturing / service / trading)
- state, district
- turnover, investment_in_plant_machinery
- employee_count
- udyam_classification (micro / small / medium)
- existing_certifications[] (ISO, ZED, etc.)
- prior_scheme_usage[]
- documents[] (linked files, OCR status)
```

Scheme

- id, name, issuing_body, category
- eligibility_rules (structured, machine-checkable where possible)
- benefit_type, benefit_amount_or_formula
- source_document_url, source_clause_reference
- last_verified_date
- application_requirements[]
- status (active / closed / superseded)

Match

- id, business_profile_id, scheme_id
- confidence_score
- citation (which rule/clause triggered the match)
- status (suggested / ca_approved / ca_rejected / applied / disbursed)

Application

- id, match_id
- draft_document_url
- missing_documents[]
- submission_method (api / manual)
- submitted_date, status, disbursed_amount, disbursed_date

SuccessFeeTransaction

- id, application_id, ca_firm_id
- fee_percentage, fee_amount, invoice_status

AuditLog

- every AI-generated claim: what was claimed, what source it cited, what confidence, who approved/rejected it, when

7. AI Agent Pipeline & Reliability Requirements

This section exists because the single biggest risk in this product category is a confident, wrong answer about money a small business owner is counting on. These are not optional engineering nice-to-haves — treat them as launch blockers.

1. **No eligibility claim without a citation.** Every match must reference the specific scheme clause that justifies it, stored and shown to the CA. If the agent can't cite it, it doesn't show it.
2. **Grounded retrieval only.** The matching agent reasons over the Scheme Knowledge Base via retrieval, not from the model's parametric memory. Scheme rules change (turnover thresholds, loan caps, sub-scheme names) — the model must never "recall" a rule from training data.

3. **Confidence thresholds with an honest fallback.** Below a defined confidence threshold, the correct output is "needs manual verification," not a guessed match.
 4. **Versioned knowledge base with visible freshness.** Every scheme entry shows a "last verified" date to the CA. Build a recurring process (manual or semi-automated) to re-check source documents on a schedule — schemes get revised mid-year.
 5. **Mandatory human-in-loop before filing.** No application is submitted without explicit CA approval. This is a trust requirement and a liability boundary, not just UX.
 6. **Full audit trail.** Every claim, its source, its confidence, and the human decision on it must be logged and retrievable — for the CA's own liability protection as much as yours.
 7. **Adversarial/edge-case testing before launch.** Specifically test: businesses that are borderline-eligible, schemes with recently changed thresholds, and documents with OCR errors — these are where hallucinated confidence is most likely and most damaging.
-

8. Integrations

Integration	Purpose	Notes
Udyam Registration	Pull/verify business classification	Check current API/portal access terms
GSTN	Turnover/filing verification	May require GSP/ASP partnership for API access
DigiLocker	Document verification	Useful for certificate authenticity
WhatsApp Business API	Notification digest to CA	Not the primary interface in v1 — CAs work at a desk with documents, this is a nudge channel only
Payment/invoicing	Success-fee billing	Razorpay/similar is sufficient for v1

Explicitly not integrated in v1: OCEN/Account Aggregator (that's a credit-agent product, different from subsidy capture), GeM/tender APIs, ONDC.

9. Feature List — V1 (Prioritized)

Must-have (launch blockers):

1. Client onboarding — document upload + manual entry fallback
2. Business profile auto-extraction (OCR) with manual correction
3. Scheme Knowledge Base for the chosen state + 1-2 categories, fully cited and versioned
4. Eligibility matching with citation and confidence score
5. CA approval/rejection workflow for each match
6. Application drafting with missing-document checklist
7. Manual filing support (submission-ready package export) — API filing is a stretch goal, not a blocker
8. Status tracking (matched → drafted → submitted → disbursed)
9. Success fee calculation and basic invoicing
10. Audit log (Section 7, item 6)

Should-have (can slip past launch without blocking it):

- WhatsApp notification digest
- Multi-client bulk view / sort by highest-value match
- Basic analytics for the CA (total value recovered across clients)

Explicitly deferred — see Section 13.

10. UI/UX Principles

10.1 Accessibility Requirements

- WCAG 2.1 AA as the baseline, not an afterthought.
- Colour is never the only signal for status (approved/rejected/pending) — pair colour with icon shape and text label, so the product works for colour-blind users without a second look.
- All interactive elements keyboard-navigable; screen-reader labels on every icon-only control.
- Minimum 4.5:1 text contrast; no text-in-image for anything functional.
- Font sizes adjustable; no fixed-px text that breaks browser zoom.

10.2 Simplicity Principles

- One primary action per screen. If a screen has more than one obvious next step, split it.

- Progressive disclosure: show the match and the ₹ value first; citation and confidence detail are one click away, not upfront clutter.
 - No vanity dashboards. Every number on screen should change what the CA does next, or it doesn't belong there.
 - Default to the highest-value, highest-confidence matches at the top of every list — CAs are time-poor, optimize for their first 10 seconds on a screen.
-

11. Screen-by-Screen Specification

1. Login / Firm Setup

- Firm name, primary contact, state of practice (sets default scheme scope)

2. Client List

- Table: client name, sector, # of matched schemes, total potential ₹ value, status
- "Add Client" button — single form: business name, Udyam number or GSTIN, sector, upload documents (or skip and enter manually)

3. Client Profile Detail

- Auto-extracted fields shown with an "edit" affordance on each (never silently trust OCR)
- Document list with extraction status (success / needs review)

4. Eligibility Matches

- Card per match: scheme name, estimated ₹ benefit, confidence badge (High/Medium/Needs Review — never colour-only), one-line reason, "View citation" expandable, Approve / Reject buttons

5. Application Workspace

- Per approved match: drafted application preview, missing-document checklist, "Mark ready to file" action, manual filing instructions or API-submit button

6. Status Tracker

- Kanban-style: Matched → Drafted → Submitted → Approved → Disbursed, per client

7. Success Fee Ledger

- Per client: disbursed amount, fee %, fee amount, invoice status

- New matches, approaching deadlines, status changes; WhatsApp opt-in toggle
-

12. Suggested Tech Stack

- **Backend:** Python (FastAPI) — plays well with RAG/LLM orchestration tooling
- **Database:** PostgreSQL for structured entities; a vector store (pgvector is sufficient at this scale) for scheme-document retrieval
- **Frontend:** React web dashboard (no native app needed for v1 — the CA works at a desk)
- **Document processing:** OCR via an existing service rather than building in-house at this stage
- **LLM layer:** any strong current model via API, constrained to RAG over the Scheme Knowledge Base per Section 7 — the model choice matters far less than the grounding architecture around it
- **Hosting:** any major cloud; nothing about this workload demands anything exotic at v1 scale

This is a suggestion, not a mandate — adapt to whatever your team already knows well.

13. Explicit Out-of-Scope — Phase 2+ Backlog

Listed for roadmap visibility only. **None of this gets built until Section 3's Gate 1 is hit**, and even then, add based on what CA users actually ask for repeatedly — not in this order by default.

- Additional scheme categories / additional states
- Compliance-deadline monitoring agent (GST/MCA filing reminders)
- Tender/GeM matching agent
- Procurement/supplier discovery agent
- Voice-first "business journal" input layer
- Persistent "business memory" beyond what's needed for scheme matching
- Credit/lending agent (this is a regulated OCEN/Account-Aggregator business, not a feature)
- Community features (founder groups, matchmaking, expert booking)
- Multi-agent "board meeting" simulation

- Business Knowledge Graph — only becomes real once real usage data exists to build it from
-

14. Open Questions for the Production Team

1. Which state and which 1-2 scheme categories are we launching with?
2. Do we have (or can we get) API access to Udyam/GSTN, or is v1 manual-document-based only?
3. Who owns keeping the Scheme Knowledge Base current — a person, a process, or both?
4. What's the actual success-fee percentage and how is it contractually structured with pilot CA firms?
5. Who are the first 5 CA firms for the pilot, and how are they being recruited?